

Periodic Task Layer for FreeRTOS

GROUP 3

Sandro Marghella s358481

Pietro Alberto Levo s354939

Cristina Rizzo s361451

Gianluca Riva Governanda s359182

How is the code structured?



xPTL_CreateTask()

The function is responsible for the task superstructure used by the scheduler implemented in the project.

```
PTL_Status_t xPTL_CreateTask( const PTL_TaskConfig_t *xCfg,
PTL_TaskHandle_t **pxHandle )
{
    BaseType_t xTaskStatus;
    struct PTL_TCB *pxTCB;

    /* Ensure PTL_Init was called first */
    if ( xInitialized == pdFALSE )
    {
        return PTL_ERROR;
    }

    /* Validate input parameters */
    if ( prvPTL_ValidateTaskConfiguration(xCfg) == pdFALSE )
    {
        return PTL_INVALID_PARAM;
    }

    /* Check if the pre-allocated table has space */
    if ( ucTaskCount >= xSchedulerCfg.maxTasks ) {
        return PTL_NO_MEMORY;
    }

    /* Pick the next available slot in the table */
    pxTCB = &pxTaskTable[ ucTaskCount ];

    /* Initialize internal TCB */
    pxTCB->xCfg = *xCfg;
    pxTCB->ulJobCounter = 0;
    pxTCB->ulDeadlinesMisses = 0;
    pxTCB->ulOVERRUNS = 0;
```

xPTL_CreateTask()

- Validates the task configuration and checks scheduler capacity.
- Initializes the task's internal control structure.
- Creates the corresponding FreeRTOS task.
- Stores the task handle and returns it to the caller.

```
/* if deadline was not specified then Deadline = Period */
if ( pxTCB->xCfg.xDeadline == 0 )
{
    pxTCB->xCfg.xDeadline = pxTCB->xCfg.xPeriod;
}

/* Create the FreeRTOS task */
xTaskStatus = xTaskCreate(
    prvPTL_TaskWrap,
    pxTCB->xCfg.pcTaskName,
    xCfg->usStack_size,
    pxTCB,
    xCfg->xPriority,
    &pxTCB->xHandle
);

pxTCB->xNextRelease = 0;

if ( xTaskStatus != pdPASS )
{
    return PTL_NO_MEMORY;
}

/* Export the xHandle as an opaque pointer */
if ( pxHandle != NULL )
{
    *pxHandle = ( PTL_TaskHandle_t * )pxTCB;
}

ucTaskCount++;
return PTL_OK;
}
```

Task's struct fields

```
struct PTL_TCB
{
    TaskHandle_t xHandle;
    PTL_TaskConfig_t xCfg;
    TickType_t xReleaseTime;
    TickType_t xNextRelease;
    TickType_t xAbsoluteDeadline;
    TickType_t xMaxReleaseJitter;
    uint32_t ulJobCounter;
    uint32_t ulPendingJobs;
    uint32_t ulDeadlinesMisses;
    uint32_t ulOverruns;
    volatile BaseType_t xJobRunning;
};
```

- **xHandle**: handle for interacting with FreeRTOS APIs;
- **xCfg**: link to stack static configuration;
- **xReleaseTime**: tick in which the job was released;
- **xNextRelease**: tick in which the new job will be released;
- **xAbsoluteDeadline**: $xReleaseTime + xDeadline$;
- **xMaxReleaseJitter**: delay between **xNextRelease** and the actual release;
- **ulJobCounter**: total number of jobs released;
- **ulPendingJobs**: total number accumulated and awaiting execution;
- **ulDeadlinesMisses**: total misses number;
- **ulOverruns**: total overruns number;
- **xJobRunning**: running jobs.

Task's struct fields

```
typedef struct {  
    const char *pcTaskName;  
    TaskFunction_t PTL_JobFunction_t;  
    uint16_t usStack_size;  
    UBaseType_t xPriority;  
    TickType_t xPeriod;  
    TickType_t xDeadline;  
    TickType_t xPhase;  
    void* pArg;  
} PTL_TaskConfig_t;
```

- **pcTaskName**: task's name;
- **PTL_JobFunction_t**: address of task's function;
- **usStack_size**: size of the task in memory;
- **xPriority**: task's priority;
- **xPeriod**: task cyclic activation period;
- **xDeadline**: maximum time allowed for the task to complete its execution;
- **xPhase**: time offset from the task's very first launch;
- **pArg**: task's arguments.

xPTL_Init()

This function prepares the various elements needed to start the scheduler:

- allocates the table containing the created tasks;
- initializes the scheduler;
- starts the xTaskGetTickCount() and saves when it begins in xInitTickCount().

```
PTL_Status_t xPTL_Init( const SchedulerConfig_t *xCfg )
{
    if ( xCfg == NULL || xCfg->maxTasks == 0 )
    {
        return PTL_INVALID_PARAM;
    }

    /* Prevent double initialization */
    if ( xInitialized == pdTRUE )
    {
        return PTL_ERROR;
    }

    /* Dynamic allocation using FreeRTOS Heap */
    pxTaskTable = pvPortMalloc( sizeof( struct TCB ) * xCfg->maxTasks );

    if ( pxTaskTable == NULL )
    {
        return PTL_NO_MEMORY;
    }

    /* Function to clear memory to avoid garbage values */
    memset( pxTaskTable, 0, sizeof( struct PTL_TCB ) * xCfg->maxTasks );

    xSchedulerCfg = *xCfg;
    ucTaskCount = 0;
    xInitialized = pdTRUE;

    memset( xLogBuffer, 0, sizeof( xLogBuffer ) );
    ulLogHead = 0;
    ulLogTail = 0;

    xInitTickCount = xTaskGetTickCount();

    return PTL_OK;
}
```

Scheduler's struct fields

```
typedef struct {  
    uint8_t maxTasks;  
    PTL_Policy_t xPolicy;  
    BaseType_t xTraceEnabled;  
} SchedulerConfig_t;
```

- **maxTasks:** maximum number of tasks;
- **xPolicy:** overrun strategy adopted;
- **xTraceEnabled:** enables dynamic tracking of system events in the log buffer.

xPTL_Start()

The function prepares the tasks so that they are synchronized to a common time base.

It:

- sets the time base;
- calculates initial releases, resets the jobs, and marks them as not running;
- starts the scheduler.

```
PTL_Status_t xPTL_Start( void )
{
    xInitTickCount = xTaskGetTickCount();

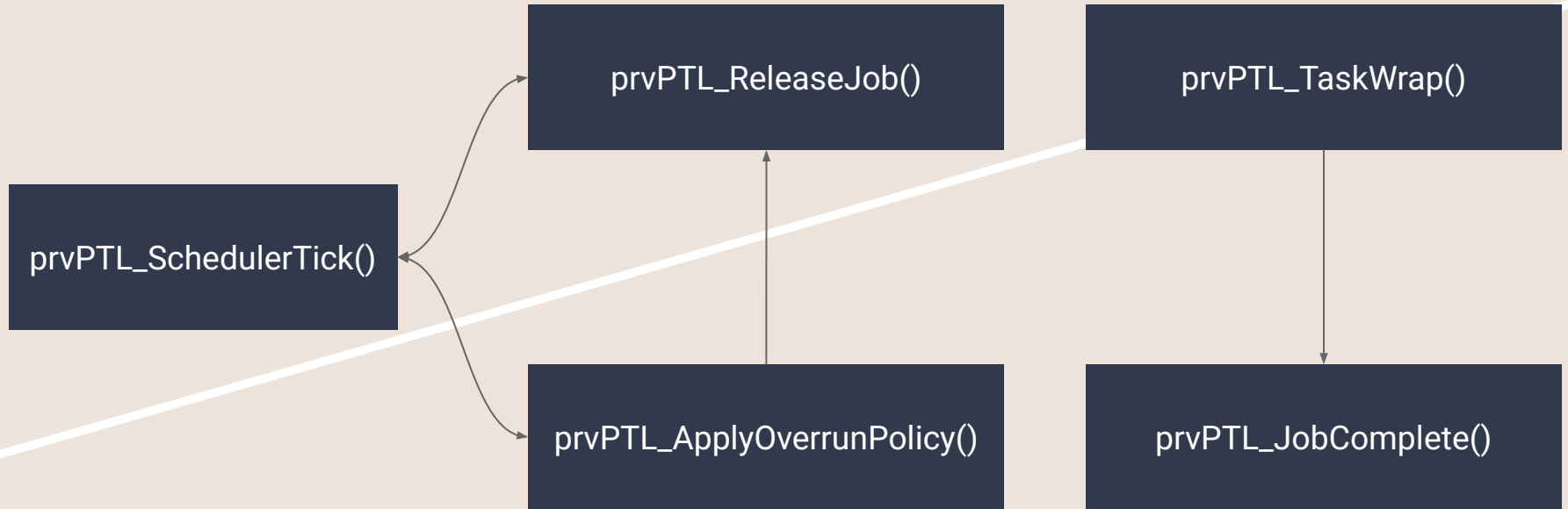
    if ( xInitialized == pdFALSE )
    {
        return PTL_ERROR;
    }

    for( int i = 0; i < ucTaskCount; i++ )
    {
        struct PTL_TCB *pxTCB = &pxTaskTable[ i ];
        pxTCB->xNextRelease = pxTCB->xCfg.xPhase;
        pxTCB->ulJobCounter = 0;
        pxTCB->xJobRunning = pdFALSE;
    }

    vTaskStartScheduler();

    return PTL_OK; /* Code should never reach this point */
}
```

How is the scheduler structured?



prvPTL_SchedulerTick()

This function is the heart of the scheduler.

It:

- manages the release of the tasks's new jobs;
- handles cases where a task exceeds its deadline;
- updates how much its takes to manage the task's jobs.

```
static void prvPTL_SchedulerTick( void )
{
    TickType_t xStart = xTaskGetTickCountFromISR();
    TickType_t xNow = xTaskGetTickCountFromISR() - xInitTickCount;
    BaseType_t pxHigherWoken = pdFALSE;

    if ( xInitialized == pdFALSE ) {
        xOverheadTicks += ( xTaskGetTickCountFromISR() - xStart );
        return;
    }

    for ( uint8_t ux = 0; ux < ucTaskCount; ux++ )
    {
        struct PTL_TCB *pxTCB = &pxTaskTable[ ux ];

        if ( xNow >= pxTCB->xNextRelease )
        {
            xIsSystemIdle = pdFALSE;
            if ( pxTCB->xJobRunning == pdTRUE )
            {
                prvPTL_ApplyOvrerrunPolicy( pxTCB, &pxHigherWoken );
            }
            else
            {
                prvPTL_ReleaseJob( pxTCB, &pxHigherWoken );
            }
        }
    }

    xOverheadTicks += ( xTaskGetTickCountFromISR() - xStart );
    portYIELD_FROM_ISR( pxHigherWoken );
}
```

prvPTL_ReleaseJob()

The function manage to release the new tasks' jobs.

It:

- calculate which tick has been reached;
- updates the xReleaseJitter and the xMaxReleaseJitter;
- updates the tasks' fields;
- updates the log for the tests.
- starts the new tasks' job.

```
static void prvPTL_ReleaseJob( struct PTL_TCB *pxTCB, BaseType_t
*pxHigherWoken )
{
    TickType_t xNow = xTaskGetTickCountFromISR() - xInitTickCount;
    if ( xNow > pxTCB->xNextRelease )
    {
        TickType_t xReleaseJitter = xNow - pxTCB->xNextRelease;
        if ( xReleaseJitter > pxTCB->xMaxReleaseJitter )
        {
            pxTCB->xMaxReleaseJitter = xReleaseJitter;
        }
    }

    UBaseType_t xSavedITStatus = taskENTER_CRITICAL_FROM_ISR();
    pxTCB->ulJobCounter++;
    pxTCB->ulPendingJobs++;
    pxTCB->xReleaseTime = pxTCB->xNextRelease;
    pxTCB->xAbsoluteDeadline = pxTCB->xReleaseTime +
pxTCB->xCfg.xDeadline;
    pxTCB->xNextRelease += pxTCB->xCfg.xPeriod;
    pxTCB->xJobRunning = pdTRUE;
    taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus );

    prvPTL_LogFromISR( PTL_EVENT_RELEASE, pxTCB,
pxTCB->ulJobCounter );
    xTaskNotifyFromISR( pxTCB->xHandle, PTL_BIT_RELEASE, eSetBits,
pxHigherWoken );
}
```

prvPTL_ApplyOverrunPolicy()

It manages all the cases of overrun:

- Overrun skip;
- Overrun kill;
- Overrun catch up.

```
static void prvPTL_ApplyOverrunPolicy( struct PTL_TCB *pxTCB, BaseType_t
*pxHigherWoken )
{
    pxTCB->ulOverruns++;
    prvPTL_LogFromISR( PTL_EVENT_OVERRUN, pxTCB, pxTCB->ulJobCounter );

    switch( xSchedulerCfg.xPolicy )
    {
        case PTL_OVERRUN_SKIP:
            /* Skip the new job; let the late one finish. */
            prvPTL_LogFromISR( PTL_EVENT_SKIP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus1 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->xNextRelease += pxTCB->xCfg.xPeriod;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus1 );
            break;
        case PTL_OVERRUN_KILL:
            /* Terminate/Suspend the running job immediately and release
the new one. */
            prvPTL_LogFromISR( PTL_EVENT_KILL, pxTCB,
pxTCB->ulJobCounter );
            xTaskNotifyFromISR( pxTCB->xHandle, PTL_BIT_ABORT, eSetBits,
pxHigherWoken );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
        case PTL_OVERRUN_CATCH_UP:
            /* release now, mark previous job missed, keep nominal
cadence. */
            prvPTL_LogFromISR( PTL_EVENT_CATCH_UP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus2 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->ulDeadlinesMisses++;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus2 );
            prvPTL_LogFromISR( PTL_EVENT_DEADLINE_MISS, pxTCB,
pxTCB->ulJobCounter );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
    }
}
```

prvPTL_ApplyOverrunPolicy()

Overrun skip:

It allows the job to finish, despite the deadline.

It:

- updates the log for the tests;
- updates the task's xNextRelease field.

```
static void prvPTL_ApplyOverrunPolicy( struct PTL_TCB *pxTCB, BaseType_t
*pxHigherWoken )
{
    pxTCB->ulOverruns++;
    prvPTL_LogFromISR( PTL_EVENT_OVERRUN, pxTCB, pxTCB->ulJobCounter );

    switch( xSchedulerCfg.xPolicy )
    {
        case PTL_OVERRUN_SKIP:
            /* Skip the new job; let the late one finish. */
            prvPTL_LogFromISR( PTL_EVENT_SKIP, pxTCB,
            pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus1 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->xNextRelease += pxTCB->xCfg.xPeriod;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus1 );
            break;
        case PTL_OVERRUN_KILL:
            /* Terminate/Suspend the running job immediately and release
            the new one. */
            prvPTL_LogFromISR( PTL_EVENT_KILL, pxTCB,
            pxTCB->ulJobCounter );
            xTaskNotifyFromISR( pxTCB->xHandle, PTL_BIT_ABORT, eSetBits,
            pxHigherWoken );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
        case PTL_OVERRUN_CATCH_UP:
            /* release now, mark previous job missed, keep nominal
            cadence. */
            prvPTL_LogFromISR( PTL_EVENT_CATCH_UP, pxTCB,
            pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus2 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->ulDeadlinesMisses++;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus2 );
            prvPTL_LogFromISR( PTL_EVENT_DEADLINE_MISS, pxTCB,
            pxTCB->ulJobCounter );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
    }
}
```

prvPTL_ApplyOverrunPolicy()

Overrun kill:

It kills the job currently running to start the next one.

It:

- updates the log for the tests;
- aborts the currently job;
- releases the new job.

```
static void prvPTL_ApplyOverrunPolicy( struct PTL_TCB *pxTCB, BaseType_t
*pxHigherWoken )
{
    pxTCB->ulOverruns++;
    prvPTL_LogFromISR( PTL_EVENT_OVERRUN, pxTCB, pxTCB->ulJobCounter );

    switch( xSchedulerCfg.xPolicy )
    {
        case PTL_OVERRUN_SKIP:
            /* Skip the new job; let the late one finish. */
            prvPTL_LogFromISR( PTL_EVENT_SKIP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus1 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->xNextRelease += pxTCB->xCfg.xPeriod;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus1 );
            break;
        case PTL_OVERRUN_KILL:
            /* Terminate/Suspend the running job immediately and release
the new one. */
            prvPTL_LogFromISR( PTL_EVENT_KILL, pxTCB,
pxTCB->ulJobCounter );
            xTaskNotifyFromISR( pxTCB->xHandle, PTL_BIT_ABORT, eSetBits,
pxHigherWoken );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
        case PTL_OVERRUN_CATCH_UP:
            /* release now, mark previous job missed, keep nominal
cadence. */
            prvPTL_LogFromISR( PTL_EVENT_CATCH_UP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus2 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->ulDeadlinesMisses++;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus2 );
            prvPTL_LogFromISR( PTL_EVENT_DEADLINE_MISS, pxTCB,
pxTCB->ulJobCounter );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
    }
}
```

prvPTL_ApplyOverrunPolicy()

Overrun catch up:

It allows the job to finish, despite the deadline but it increments the misses. it:

- updates the log for the tests;
- updates the ulDeadlinesMisses;
- releases the new job.

```
static void prvPTL_ApplyOverrunPolicy( struct PTL_TCB *pxTCB, BaseType_t
*pxHigherWoken )
{
    pxTCB->ulOverruns++;
    prvPTL_LogFromISR( PTL_EVENT_OVERRUN, pxTCB, pxTCB->ulJobCounter );

    switch( xSchedulerCfg.xPolicy )
    {
        case PTL_OVERRUN_SKIP:
            /* Skip the new job; let the late one finish. */
            prvPTL_LogFromISR( PTL_EVENT_SKIP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus1 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->xNextRelease += pxTCB->xCfg.xPeriod;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus1 );
            break;
        case PTL_OVERRUN_KILL:
            /* Terminate/Suspend the running job immediately and release
the new one. */
            prvPTL_LogFromISR( PTL_EVENT_KILL, pxTCB,
pxTCB->ulJobCounter );
            xTaskNotifyFromISR( pxTCB->xHandle, PTL_BIT_ABORT, eSetBits,
pxHigherWoken );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
        case PTL_OVERRUN_CATCH_UP:
            /* release now, mark previous job missed, keep nominal
cadence. */
            prvPTL_LogFromISR( PTL_EVENT_CATCH_UP, pxTCB,
pxTCB->ulJobCounter );
            UBaseType_t xSavedITStatus2 = taskENTER_CRITICAL_FROM_ISR();
            pxTCB->ulDeadlinesMisses++;
            taskEXIT_CRITICAL_FROM_ISR( xSavedITStatus2 );
            prvPTL_LogFromISR( PTL_EVENT_DEADLINE_MISS, pxTCB,
pxTCB->ulJobCounter );
            prvPTL_ReleaseJob( pxTCB, pxHigherWoken );
            break;
    }
}
```


prvPTL_TaskWrap()

Waiting for Job Release:

- Checks whether pending jobs are available.
- If no job is ready, the task waits for a notification.
- PTL_BIT_RELEASE wakes the task and starts a new job.
- PTL_BIT_ABORT cancels the current wait.
- Each job is associated with a unique Job ID.

```
static void prvPTL_TaskWrap( void *pvParameters )
{
    struct PTL_TCB *pxTCB = ( struct PTL_TCB * )pvParameters;
    uint32_t ulNotifiedValue;
    /* counter of local jobs */
    uint32_t ulLocalJobId = 0;

    for( ;; ){
        /* check if old jobs present*/
        taskENTER_CRITICAL();
        BaseType_t xHasWork = ( pxTCB->ulPendingJobs > 0 );
        if ( xHasWork )
        {
            ulLocalJobId = pxTCB->ulJobCounter - (
pxTCB->ulPendingJobs - 1 );
        }
        taskEXIT_CRITICAL();

        if ( !xHasWork )
        {
            xTaskNotifyWait( 0, PTL_BIT_RELEASE | PTL_BIT_ABORT,
&ulNotifiedValue, portMAX_DELAY );
            if ( ulNotifiedValue & PTL_BIT_ABORT ) {
                continue;
            }
            if ( ulNotifiedValue & PTL_BIT_RELEASE ) {
                taskENTER_CRITICAL();
                ulLocalJobId = pxTCB->ulJobCounter - (
pxTCB->ulPendingJobs - 1 );
                xHasWork = pdTRUE;
                taskEXIT_CRITICAL();
            }
        }
    }
}
```

prvPTL_TaskWrap()

Job Execution:

- Records the job start event for tracing.
- Executes the user-defined PTL_JobFunction_t.
- Calls prvPTL_JobComplete() when execution finishes.
- Updates the number of pending jobs.
- Measures the overhead introduced by the PTL.

```
if ( xHasWork )
{
    TickType_t xStart = xTaskGetTickCount();
    prvPTL_Log( PTL_EVENT_START, pxTCB, ulLocalJobId );

    xOverheadTicks += ( xTaskGetTickCount() - xStart );

    pxTCB->xCfg.PTL_JobFunction_t( pxTCB->xCfg.pArg );

    xStart = xTaskGetTickCount();
    prvPTL_JobComplete( pxTCB, ulLocalJobId );

    taskENTER_CRITICAL();
    if ( pxTCB->ulPendingJobs > 0 ) {
        pxTCB->ulPendingJobs--;
    }
    taskEXIT_CRITICAL();
    xOverheadTicks += ( xTaskGetTickCount() - xStart );
}
}
```

prvPTL_JobComplete()

The function manage the end of the tasks' jobs.
It:

- calculate which tick has been reached;
- updates the running flag if there is not overrunning;
- updates the misses in case of overrunning;
- manages also the case of overrun-kill to update the log;
- updates the log for the tests.

```
static void prvPTL_JobComplete( struct PTL_TCB *pxTCB, uint32_t
ulJobId )
{
    TickType_t xNow = xTaskGetTickCount() - xInitTickCount;
    BaseType_t xIsStale;

    taskENTER_CRITICAL();
    xIsStale = ( pxTCB->ulJobCounter != ulJobId );
    if ( !xIsStale )
    {
        pxTCB->xJobRunning = pdFALSE;
    }
    taskEXIT_CRITICAL();

    if ( xIsStale )
    {
        if ( xSchedulerCfg.xPolicy == PTL_OVERRUN_KILL )
        {
            prvPTL_Log( PTL_EVENT_ABORTED, pxTCB, ulJobId );
        }
        else
        {
            prvPTL_Log( PTL_EVENT_COMPLETE, pxTCB, ulJobId );
        }
        return;
    }

    prvPTL_Log( PTL_EVENT_COMPLETE, pxTCB, ulJobId );

    if ( xNow > pxTCB->xAbsoluteDeadline )
    {
        taskENTER_CRITICAL();
        pxTCB->ulDeadlinesMisses++;
        taskEXIT_CRITICAL();
        prvPTL_Log( PTL_EVENT_DEADLINE_MISS, pxTCB, ulJobId );
    }
}
```

Tracing

- Overview
- Structure

Tracing overview

What is it?

It is a lightweight system to monitor program execution in real-time, created specifically to handle periodic tasks in FreeRTOS.

What it is used for?

It helps determine whether tasks are working correctly and immediately spots timing issues, such as delays or missed deadlines.

How it works?

It uses a circular buffer to record data safely from both regular tasks and system interrupts.

It logs when an activity starts or finishes and signals any errors. All of these messages are then sent externally via the UART port.

Tracing Structure I

To correctly trace the events an enum `PTL_EventType_t` has been declared.

Then it is packaged in a “message” `PTL_LogEvent_t` together with:

- `pcTaskName`
- `xTimeStamp`
- `ulJobId`

```
/**
 * @brief Enum of possible events to be logged
 */
typedef enum {
    PTL_EVENT_START,
    PTL_EVENT_DEADLINE_MISS,
    PTL_EVENT_OVERRUN,
    PTL_EVENT_RELEASE,
    PTL_EVENT_KILL,
    PTL_EVENT_SKIP,
    PTL_EVENT_CATCH_UP,
    PTL_EVENT_IDLE,
    PTL_EVENT_COMPLETE,
    PTL_EVENT_ABORTED,
} PTL_EventType_t;

/**
 * @brief Struct where log messages are packaged
 */
typedef struct {
    PTL_EventType_t xType;
    const char *pcTaskName;
    TickType_t xTimeStamp;
    uint32_t ulJobId;
} PTL_LogEvent_t;
```

Tracing Structure II

The logs are stored inside a circular buffer xLogBuffer.

The functions that write to the buffer are:

- prvPTL_Log
- prvPTL_LogFromISR

While prvPTL_FlushLogBuffer reads the buffer.

```
static PTL_LogEvent_t  xLogBuffer[ LOG_BUFFER_SIZE ];
static volatile uint32_t  ulLogHead = 0;
static volatile uint32_t  ulLogTail = 0;
static volatile uint32_t  ulLogDropped = 0;

/**
 * @brief Logs a PTL event from task context.
 * Stores event type, timestamp, task name and job id into a
 * circular buffer.
 *
 * @param xType Event type to log.
 * @param pxTCB task that generated the event(for IDLE it is NULL).
 * @param ulJobId job identifier associated with the event.
 */
static void prvPTL_Log ( PTL_EventType_t xType, struct PTL_TCB
*pxTCB, uint32_t ulJobId );

static void prvPTL_LogFromISR ( PTL_EventType_t xType, struct
PTL_TCB *pxTCB, uint32_t ulJobId );

static void prvPTL_FlushLogBuffer ( void );
```

Configuration Framework

Configuration Framework

Based on a yaml file, in which the tasks are described, and a python script which checks the schedulability and generate the following files:

- include/FreeRTOSConfig.h
- include/task.h
- src/main.c
- src/task.c

```
scheduler:  
  policy: SKIP  
  max_tasks: 8  
  trace_enabled: true  
  
tasks:  
  - name: BrakeMonitor  
    period_ms: 50  
    deadline_ms: 50  
    priority: 4  
    wcet_ms: 5  
  
  - name: EngineSensor  
    period_ms: 100  
    deadline_ms: 50  
    priority: 3  
    wcet_ms: 10  
  
  - name: Dashboard  
    period_ms: 200  
    deadline_ms: 150  
    priority: 2  
    wcet_ms: 15  
  
  - name: BrakePedalSim  
    period_ms: 500  
    priority: 1  
    wcet_ms: 5
```

Configuration Framework

Commands:

- make analyze FILE=<filename>
- make generate FILE=<filename>

T=period

D=deadline

WCET=worst case execution time

R=response time

RESPONSE TIME ANALYSIS EXAMPLES:

```
$> make analyze FILE=taskset1
tests/config/venv/bin/python3 config_framework.py analyze
taskset1.yaml
tick_hz = 100000 (1 tick = 0.010 ms)
task          T(tick) D(tick) C(tick) R(tick)  result
BrakeMonitor   5000   5000   500    500    OK
EngineSensor  10000   5000   1000   1500    OK
Dashboard     20000  15000   1500   3000    OK
BrakePedalSim  50000  50000   500    3500    OK
SCHEDULABLE
```

```
$> make analyze FILE=taskset2
tests/config/venv/bin/python3 config_framework.py analyze
taskset2.yaml
tick_hz = 100000 (1 tick = 0.010 ms)
task          T(tick) D(tick) C(tick) R(tick)  result
BrakeMonitor   5000   5000   500    500    OK
EngineSensor  10000   5000   4800   5300  MISS
Dashboard     20000  15000   1500   7300    OK
BrakePedalSim  50000  50000   500    7800    OK
NOT SCHEDULABLE
```

Testing

- Testing suite
 - ◆ Tests list
 - ◆ Examples
- Documentation

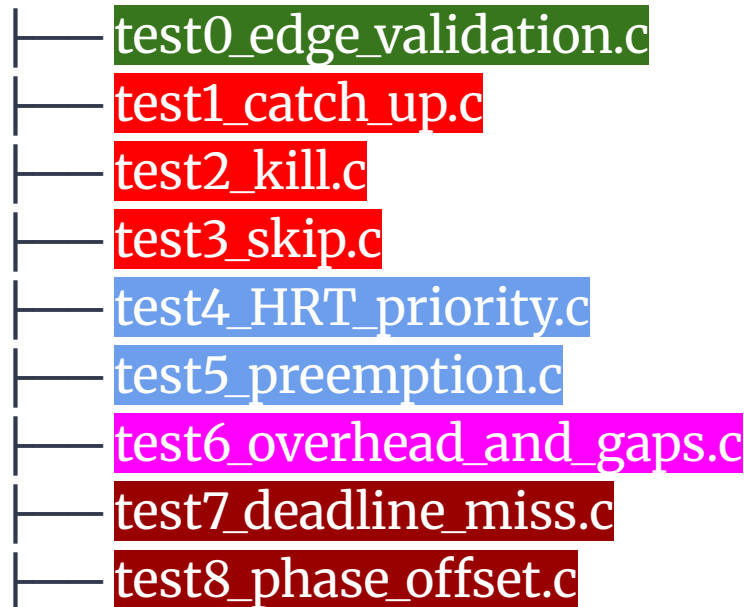
Testing

Testing phase was performed following macro categories:

- Input validation
- Overrun policy
- Scheduling correctness
- Non-functional requirements
- Feature checks and corner cases

From each test, the trace is analyzed by Python scripts that

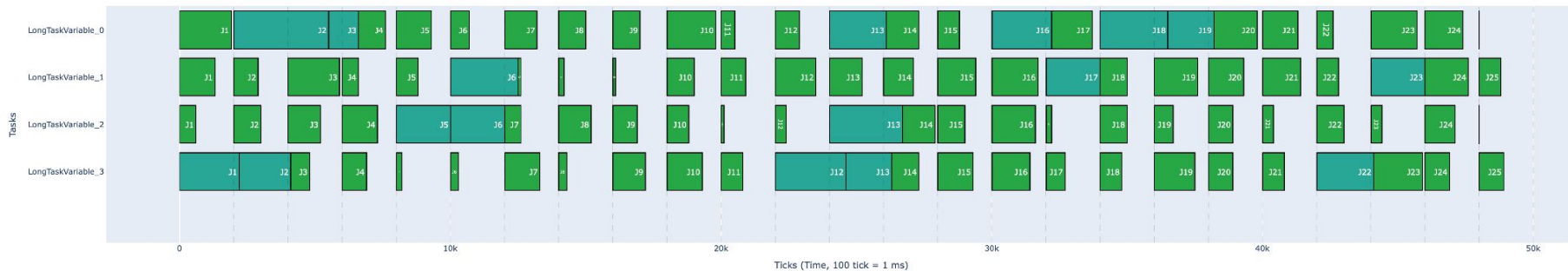
- Check their correctness and display the scheduling Gantt
- Print a .rpt file with test details



- test0_edge_validation.c
- test1_catch_up.c
- test2_kill.c
- test3_skip.c
- test4_HRT_priority.c
- test5_preemption.c
- test6_overhead_and_gaps.c
- test7_deadline_miss.c
- test8_phase_offset.c

Examples

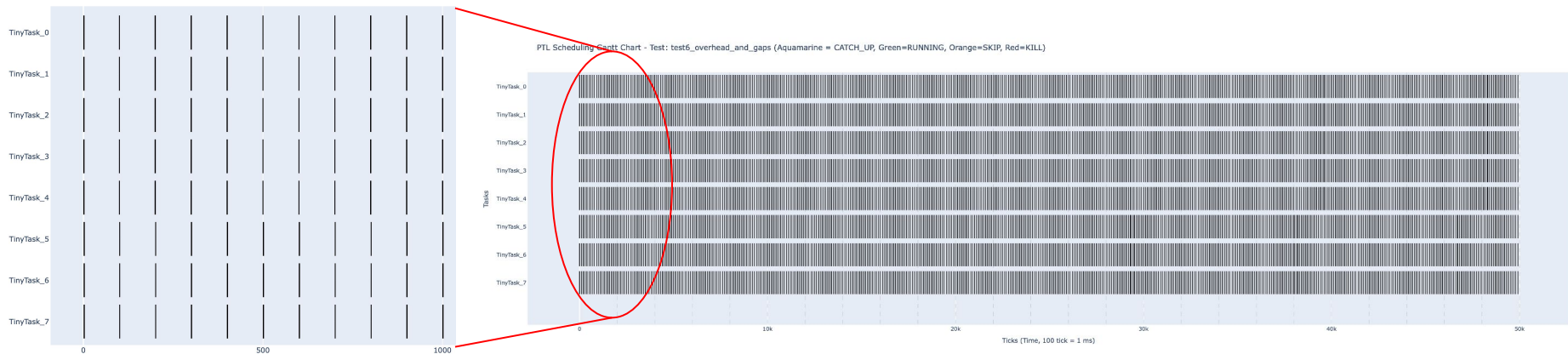
PTL Scheduling Gantt Chart - Test: test1_catch_up (Aquamarine = CATCH_UP, Green=RUNNING, Orange=SKIP, Red=KILL)



PTL Scheduling Gantt Chart - Test: test2_kill (Aquamarine = CATCH_UP, Green=RUNNING, Orange=SKIP, Red=KILL)



Examples (NFR stress)



```
1 == PTL TEST REPORT: test6_overhead_and_gaps ==  
  
[RESULT] OVERHEAD + MINIMAL GAPS: PASSED  
[TASK] TinyTask_0: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_1: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_2: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_3: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_4: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_5: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_6: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[TASK] TinyTask_7: jobs=501 misses=0 overruns=0 maxJitter=1 (OK)  
[NFR] CPU overhead: 1.00% (PASS)  
[NFR] Jitter Violation hook: PASS  
  
== OVERALL: PASSED ==
```

≤1 tick Jitter ✓

≤10% CPU overhead ✓

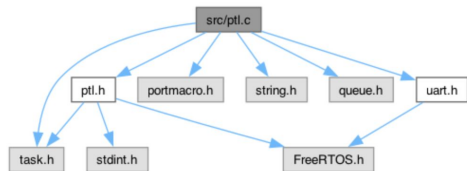
Documentation

- The documentation has been created through **Doxygen**
- Extracts documentation from the comments in the source code
- Makes relationships and cross-references easy to navigate in a complex project

ptl.c File Reference

```
#include "ptl.h"  
#include "portmacro.h"  
#include "string.h"  
#include "task.h"  
#include "queue.h"  
#include "uart.h"
```

Include dependency graph for ptl.c:



Here is a list of all struct and union fields with links to the structures/unions they belong to:

[M](#) | [P](#) | [U](#) | [X](#)

- maxTasks : SchedulerConfig_t
- pArg : PTL_TaskConfig_t
- pcTaskName : PTL_LogEvent_t, PTL_TaskConfig_t
- PTL_JobFunction_t : PTL_TaskConfig_t
- ulDeadlinesMisses : PTL_TaskStats_t, PTL_TCB
- ulJobCounter : PTL_TaskStats_t, PTL_TCB
- ulJobId : PTL_LogEvent_t
- ulOverruns : PTL_TaskStats_t, PTL_TCB
- ulPendingJobs : PTL_TCB
- usStack_size : PTL_TaskConfig_t
- xAbsoluteDeadline : PTL_TCB
- xCfg : PTL_TCB
- xDeadline : PTL_TaskConfig_t
- xHandle : PTL_TCB
- xJobRunning : PTL_TCB
- xMaxReleaseJitter : PTL_TaskStats_t, PTL_TCB
- xNextRelease : PTL_TCB
- xPeriod : PTL_TaskConfig_t
- xPhase : PTL_TaskConfig_t
- xPolicy : SchedulerConfig_t
- xPriority : PTL_TaskConfig_t
- xReleaseTime : PTL_TCB
- xTimeStamp : PTL_LogEvent_t
- xTraceEnabled : SchedulerConfig_t
- xType : PTL_LogEvent_t

